

Web Uygulamalarında Brute-Force

Eğitim Özeti

"Web Uygulamalarında Brute-Force" eğitimi, siber güvenlik uzmanlarına web uygulamalarında brute-force saldırılarını gerçekleştirme ve bu saldırılara karşı korunma yöntemlerini öğretmeyi amaçlamaktadır. Bu kapsamlı eğitim, brute-force saldırılarının temel prensiplerini ve çeşitli fuzzing tekniklerini detaylı bir şekilde ele alır.

Eğitim içeriğinde, dizin fuzzing, sayfa fuzzing, alt alan adı fuzzing, sanal ana bilgisayar (vhost) fuzzing, GET ve POST parametre fuzzing gibi konular işlenecektir. Ayrıca, CAPTCHA bypass yöntemleri ve brute-force saldırılarını etkili bir şekilde gerçekleştirme teknikleri üzerinde durulacaktır. Her bölümde, ilgili brute-force tekniklerinin nasıl uygulandığı ve web uygulamalarında nasıl istismar edilebileceği pratik örneklerle açıklanacaktır. Eğitimin sonunda, brute-force saldırılarına karşı alınabilecek savunma önlemleri ve en iyi uygulamalar hakkında bilgi sahibi olacaksınız.

Giriş

Brute Force saldırıları, siber güvenlikte yaygın olarak karşılaşılan ve basit ama etkili bir saldırı türüdür. Bir saldırganın hedef sistemde doğru kimlik bilgilerini veya yapılandırma ayarlarını bulmak için sistematik olarak birçok farklı kombinasyonu denemesi anlamına gelir.



Görseldeki mantığı en basit haliyle bir **"Anahtarcı Hikayesi"** gibi düşünebilirsin. Elinde dev bir anahtarlık olan bir hırsız var ve kapıyı açana kadar her anahtarı tek tek, bıkmadan deniyor

Nedir Bu Brute Force?

Brute Force, bir saldırganın doğru kullanıcı adını, şifreyi veya gizli bir yapılandırma ayarını bulmak için **deneme-yanılma** yöntemini kullanmasıdır. Teknik bir deha gerektirmez; daha çok sabır ve işlemci gücü gerektirir.

Yaygın Brute Force Türü: Dictionary Attack (Sözlük Saldırısı)

Brute Force'un "akıllı" kuzeni diyebiliriz. Rastgele karakterler denemek yerine, insanların en çok kullandığı kelimelere odaklanır.

Tanım

Saldırganlar, içerisinde milyonlarca yaygın parolanın bulunduğu özel listeleri (wordlists) kullanırlar. Bu listeler; "123456", "password", "admin" gibi klasiklerin yanı sıra, sızdırılmış veri tabanlarından elde edilen gerçek parolaları da içerir.

Çalışma Mekanizması

Bir robotun, elindeki devasa bir sözlüğü saniyeler içinde tarayıp uygun anahtarı bulmaya çalıştığını hayal et. Teknik olarak bu işlem genellikle HTTP istekleri üzerinden yürütülür.

Örnek Senaryo: Saldırgan, hedef sitenin giriş paneline şu şekilde otomatik istekler gönderir:

- **Deneme 1:** <https://example.com/login?username=admin&password=123456>
❌ (Hatalı)
- **Deneme 2:** <https://example.com/login?username=admin&password=password>
❌ (Hatalı)
- **Deneme 3:** <https://example.com/login?username=admin&password=qwerty>
✅ (Başarılı!)

Siber_Security Notu

Neden Tehlikeli? İnsanlar karmaşık karakterler yerine hatırlaması kolay "kelimeler" seçmeye meyillidir. Bir bilgisayar için 100 milyon kelimelik bir listeyi denemek sadece birkaç dakika sürer. Eğer şifreniz bir sözlükte geçiyorsa, hacker için sadece bir "zaman" meselesisinizdir.

Savunma Yöntemleri: Güçlü parola politikaları, çok faktörlü kimlik doğrulama (MFA) ve parola denemeleri için limitler koymak.

2. Tüm Kombinasyonların Denenmesi (Brute Force Proper)

Bu yöntem, siber dünyada "balyozla kapı kırmak" gibidir. Hiçbir tahmin yürütmeden, evrendeki tüm olasılıkları tek tek dener.

Tanım

Saldırgan; harfler (a-z, A-Z), rakamlar (0-9) ve özel karakterlerden (!, @, #, vb.) oluşabilecek **her türlü kombinasyonu** sistem kabul edene kadar dener.

Çalışma Mekanizması

Bu saldırı türü tamamen bir **olasılık ve matematik** savaşıdır. Saldırganın yazılımı şu sırayı izler:

1. a, b, c... (Tek haneliler biter)
2. aa, ab, ac... (Çift haneliler biter)
3. aaa, aab... ve bu süreç şifre bulunana kadar devam eder.

Kritik Bilgi: Bu yöntem teorik olarak **her şifreyi kırabilir**. Ancak şifre ne kadar uzunsa, deneme süresi katlanarak artar. 8 karakterli basit bir şifre saniyeler içinde kırılırken, 16 karakterli karmaşık bir şifre için gereken süre **yüzlerce yılı** bulabilir.

Savunma Hattı: Kaleyi Nasıl Koruruz?

Saldırganın bu sonsuz deneme döngüsünü durdurmak için üç temel silahımız var:

1. **Parola Karmaşıklığı & Uzunluğu:** Şifreyi sadece 2 karakter uzatmak bile, saldırganın işini 100 kat zorlaştırabilir. Matematiksel olarak olasılık havuzunu (Entropy) genişletiyoruz.
2. **Oturum Sınırlaması (Rate Limiting):** "Dur bakalım!" diyoruz. 5 kez yanlış giren bir IP adresini 30 dakika engellediğimizde, saldırganın saniyede binlerce deneme yapma gücünü elinden alıyoruz.
3. **CAPTCHA:** Bilgisayarlar şifre deneyebilir ama "trafik ışıklarını seçmekte" hala bizden kötüler. Otomatik botları bu şekilde kapı dışarı ediyoruz.

3. Credential Stuffing (Kimlik Bilgisi Doldurma)

Bu yöntem, "Bir kapının anahtarını bulduysan, mahalledeki tüm kapıları dene" mantığıyla çalışır. Saldırgan, kullanıcıların **şifre tekrarı yapma alışkanlığını** sömürür.

Tanım

Daha önce başka bir platformda (örneğin eski bir oyun sitesi veya alışveriş sitesi) yaşanmış veri ihlalleri sonucu internete sızan kullanıcı adı ve parola ikililerini alıp, bu bilgileri popüler sitelerde (Instagram, Banka, E-devlet vb.) deneme işlemidir.

Çalışma Mekanizması

Saldırganlar "Combo List" adı verilen devasa listeler kullanır. Bu listeler sızdırılmış e-posta ve şifre ikililerinden oluşur.

Örnek Senaryo:

1. **Sızıntı:** A sitesi hacklenir ve ahmet@mail.com : Papatya123 bilgisi sızar.
2. **Saldırı:** Hacker, bu ikiliyi alır ve otomatik bir botla B sitesinde, C sitesinde ve D sitesinde denemeye başlar.
3. **Sonuç:** Ahmet Bey, her yerde aynı şifreyi kullanıyorsa, hacker tek bir sızıntıyla tüm dijital hayatına giriş yapar.

Savunma Hattı: "Tek Şifre Faciası"ndan Korunma

Bu saldırı türü, klasik Brute Force korumalarını (hız sınırlama vb.) bazen aşabilir çünkü denenen şifreler zaten "doğru" veya "gerçekçi" şifrelerdir.

- **MFA (Çok Faktörlü Doğrulama):** En kritik savunmadır. Hacker şifrenizi bilse bile, telefonunuza gelen onay kodunu aşamaz.

Brute Force Saldırıların Etkileri

- **Hesap Ele Geçirme:** Saldırgan, doğru kimlik bilgilerini bulduğunda kurbanın hesabını tamamen kontrol edebilir ve kişisel verileri veya hassas bilgileri ele geçirebilir.
- **Hizmet Kesintisi:** Sürekli deneme saldırıları uygulamanın performansını etkileyebilir veya tüm sistemi işlevsiz hale getirebilir.
- **Yetkisiz Erişim:** Saldırgan, brute force yoluyla elde ettiği bilgilerle yetkisiz işlemler gerçekleştirebilir veya şirket içi verilere erişebilir.

Dizin Tarama (Directory Fuzzing / Enumeration)

Bir web sitesinin sadece görünen sayfalarından ibaret olduğunu sanıyorsanız, yanılıyorsunuz. Dizin tarama, buzdağının görünmeyen kısmını keşfetme sanatıdır.

Tanım

Dizin tarama, bir web sunucusunda halka açık olarak linklenmemiş ancak sunucuda fiziksel olarak var olan gizli klasörlerin ve dosyaların (örneğin: /admin, /backup, /config.php) tespit edilmesidir.

Çalışma Mekanizması

Saldırganlar, binlerce klasör isminin bulunduğu "Wordlist" dosyalarını kullanarak otomatik araçlarla (örneğin: **ffuf**, **Dirb**, **Gobuster**) sunucuya istek gönderirler.

Süreç Şöyle İşler:

1. **İstek:** Saldırganın aracı sorar: <https://site.com/test> var mı?
2. **Yanıt:** Sunucu 404 Not Found derse, o dizin yoktur.
3. **Keşif:** Eğer sunucu 200 OK veya 403 Forbidden dönerse, saldırgan orada bir şey olduğunu anlar.

Dizin Tarama Araçları: Hızın Efendisi "ffuf"

Dizin tarama işlemini manuel olarak yapmak imkansızdır. İşte bu noktada devreye giren en popüler araçlardan biri olan **ffuf (Fuzz Faster U Fool)**, adının hakkını vererek inanılmaz bir hızla sonuç üretir.

ffuf Nedir?

Go diliyle yazılmış, hafif ama çok güçlü bir "fuzzing" aracıdır. Sadece dizinleri değil; aynı zamanda parametreleri, dosya uzantılarını ve HTTP başlıklarını da tarayabilir.

Temel Kullanım (Cheat Sheet)

En yaygın kullanım senaryosu, sunucuda gizli bir klasör aramaktır:

```
ffuf -u https://example.com/FUZZ -w /path/to/wordlist.txt
```

Parametrelerin Sırrı:

- **-u (URL):** Hedef adresi belirtir. Buradaki FUZZ kelimesi bir "yer tutucu"dur. ffuf, kelime listesindeki her satırı bu FUZZ yazan yere koyarak dener.
- **-w (Wordlist):** Saldırganın elindeki "olasılıklar listesi"nin (sözlük) yolunu gösterir.



2. Gobuster: Hedefe Kilitlenen Tarayıcı

Gobuster, tıpkı ffuf gibi Go diliyle geliştirilmiştir. Ancak Gobuster, özellikle dizin ve DNS tarama konularında daha "nokta atışı" ve yapılandırılmış bir deneyim sunmasıyla bilinir.



Gobuster Nedir?

Web sunucularındaki gizli dizinleri, dosyaları ve hatta alt alan adlarını (subdomain) bulmak için kullanılan yüksek performanslı bir araçtır. Rekabetin yüksek olduğu CTF (Capture The Flag) platformlarında hızı nedeniyle sıkça tercih edilir.



Temel Kullanım

Basit bir dizin taraması başlatmak için şu komut yeterlidir:

Bash

```
gobuster dir -u https://example.com -w /path/to/wordlist.txt
```

Komut Parçaları:

- **dir:** Gobuster'a "Dizin/Dosya Tarama" modunda çalışmasını söyler (Diğer popüler modlar: dns, vhost).
- **-u (URL):** Hedef web sitesinin adresini belirtir.
- **-w (Wordlist):** Denenecek kelimelerin bulunduğu listenin yoludur.



Gobuster'ın Avantajları

1. **Çok Yönlülük:** Sadece dizin taramaz; alt alan adlarını (dns modu) ve sanal hostları da (vhost) keşfedebilir.
2. **Uzantı Belirleme:** -x .php,.html,.txt gibi bir parametreyle sadece klasörleri değil, belirli türdeki gizli dosyaları da bulmanızı sağlar.
3. **Temiz Çıktı:** Sonuçları çok düzenli bir şekilde listeler, bu da raporlama aşamasında işinizi kolaylaştırır.



Siber_Security Profesyonel Notu

"ffuf ile Gobuster arasındaki temel fark şudur: **ffuf** her şeyi (parametreler, başlıklar vb.) tarayabilen genel bir 'fuzzer' iken, **Gobuster** dizin ve DNS keşfi üzerine özelleşmiş daha odaklı bir araçtır. Bir pentester'in çantasında her ikisinin de olması şarttır!"



Sayfa Tarama (Page Fuzzing)

Dizinleri (klasörleri) bulduk, peki ya içindeki gizli dosyalar? Sayfa tarama, bir web sitesindeki "görünmez" dosyaları ve API uç noktalarını (endpoints) ortaya çıkarma işlemidir.



Tanım

Sayfa tarama, bir saldırganın sunucuda var olan ama link verilmemiş sayfaları (örn: config.php, test.html, v1/api/users) bulmak için yaptığı bir brute-force işlemidir. Amaç, korumasız bırakılmış veya unutulmuş dosyalar üzerinden sisteme sızmadır.



Çalışma Mekanizması

Saldırganlar, dosya uzantılarını (.php, .asp, .jsp, .txt) tahmin ederek hedef URL'nin sonuna eklerler.

Örnek Senaryo: Bir saldırgan ffuf kullanarak şu sonuçlara ulaşabilir:

- <https://site.com/admin.php> -> (Bulundu!)
- https://site.com/db_backup.sql -> (Kritik Sızıntı!)
- <https://site.com/internal.html> -> (Hassas Bilgi!)

Araç: ffuf (Fuzz Faster U Fool)

Daha önce dizin taramada gördüğümüz **ffuf**, sayfa tarama konusunda da bir numaradır. Özellikle belirli uzantıları taramak için şu şekilde kullanılır:

```
ffuf -u https://example.com/FUZZ.php -w /path/to/wordlist.txt
```

Bu komut, listedeki her kelimenin sonuna .php ekleyerek sunucuda o isimde bir dosya olup olmadığını kontrol eder.

Gobuster ile Sayfa Avcılığı: Uzantıların Gücü

Dizin tararken karşımıza çıkan "boş klasörlerden" bıktıysanız, Gobuster'ın **uzantı belirleme** özelliği tam size göre.

Page Fuzzing'de Gobuster Farkı

Normal bir tarama sadece /admin veya /backup gibi klasörleri ararken, -x parametresi sayesinde Gobuster her kelimeyi bir dosya ismiymiş gibi test eder.

Gelişmiş Kullanım Örneği

Eğer hedef sistemin PHP veya ASP ile yazıldığını biliyorsanız, nokta atışı yapmak için şu komutu kullanırsınız:

```
gobuster dir -u https://example.com -w /path/to/wordlist.txt -x php,html,asp
```

Parametre Analizi:

- **-dir:** Dizin veya sayfa taraması yapar.

- **-u:** URL'yi belirtir.
- **-w:** Kelime listesi yolunu belirtir.
- **-x:** Sayfa uzantılarını belirtir (örneğin php, html, asp).

Örnek Senaryo ve Çıktı

Kelime listesinde sadece admin kelimesi olsa bile, -x php,html sayesinde Gobuster şu sayfaları tek tek dener:

1. example.com/admin.php  (Bulunabilir!)
2. example.com/admin.html  (Bulunabilir!)

BÜYÜK ÖZET: Siber_Security Yol Haritası

1. **Saldırı Türleri:** Brute Force (Kaba Kuvvet), Dictionary (Sözlük) ve Credential Stuffing (Emanet Bilgi) ile kapıları zorladık.
2. **Keşif (Fuzzing):** Directory (Dizin) ve Page (Sayfa) tarama ile sistemin haritasını çıkardık.
3. **Cephanelik:** ffuf ile hız kazandık, gobuster ile hedef odaklı taramalar yaptık.
4. **Savunma:** MFA, Rate Limiting ve güçlü şifreleme ile kaleyi korumayı öğrendik.

Sub-domain Tarama (Alt Alan Adı Keşfi)

Bir siber saldırgan için ana domain (örneğin: site.com) çok iyi korunuyor olabilir. Ancak alt alan adları (örneğin: test.site.com veya dev.site.com) genellikle daha zayıf güvenlik önlemlerine sahiptir.

Tanım

Sub-domain tarama, bir hedef domaine bağlı olan tüm alt alan adlarını (sub-domain) tespit etme işlemidir. Şirketler genellikle geliştirme aşamasındaki uygulamaları, eski veritabanlarını veya yönetim panellerini alt alan adlarında unuturlar.

⚙️ Çalışma Mekanizması: DNS Fuzzing

Saldırganlar, binlerce olası sub-domain isminin (blog, dev, vpn, mail, api vb.) bulunduğu listeleri kullanarak DNS sorguları yaparlar.

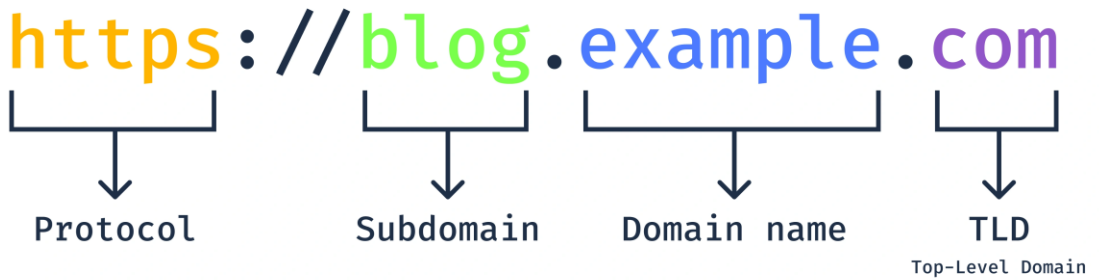
- **Sorgu:** "Sistemde dev.hedef.com adında bir kayıt var mı?"
- **Yanıt:** Eğer DNS sunucusu bir IP adresi döndürürse, o sub-domain **canlıdır** ve saldırı için yeni bir kapı açılmıştır.

⚠️ Neden Kritik Bir Tehdit?

Sub-domainler genellikle şu "gizli hazineleri" barındırır:

- **Unutulmuş Geliştirme Ortamları:** test.site.com gibi sayfalar, güvenlik duvarı arkasında olmayan zayıf kodlar içerebilir.
- **Eski Yönetim Panelleri:** admin-panel.site.com gibi sızdırılmaya müsait giriş noktaları.
- **Gölge IT (Shadow IT):** Bilgi işlem biriminin haberi olmadan açılmış ve güncellenmemiş servisler.

🌐 Bir URL'nin Anatomisi



Sub-domain Tarama Araçları: ffuf (Fuzz Faster U Fool)

Sub-domain tarama sürecinde **ffuf**, hızı ve esnekliği sayesinde siber güvenlik uzmanlarının en çok tercih ettiği araçlardan biridir. Sadece DNS kayıtlarını değil, doğrudan sunucu başlıklarını hedef alarak derinlemesine keşif yapmanıza olanak tanır.

ffuf ile Sub-domain Keşfi (Komut Yapısı)

Aşağıdaki komut, hedef domain üzerindeki alt alan adlarını tespit etmek için kullanılır:

```
ffuf -u https://FUZZ.example.com -w /path/to/wordlist.txt -H "Host FUZZ.example.com"
```

- **-u:** Alt alan adı ile birlikte URL'yi belirtir. FUZZ anahtar kelimesi test noktası olarak kullanılır.
- **-w:** Kelime listesini belirtir. wordlist.txt içinde denenecek sub-domain isimleri yer alır.
- **-H:** Alt alan adı ile birlikte gönderilecek HTTP başlığını belirtir.

OSINT Odaklı Keşif: Sublist3r

Sublist3r, hedef bir alan adına ait alt alan adlarını (sub-domains) tespit etmek için tasarlanmış, Python tabanlı popüler bir araçtır. Birçok farklı kaynaktan (Google, Bing, Baidu gibi arama motorları ve Netcraft, VirusTotal gibi servisler) veri toplayarak kapsamlı bir sonuç sunar.

Sublist3r Kullanımı

Hızlı ve etkili bir tarama için şu komut yapısı kullanılır:

```
sublist3r -d example.com -o subdomains.txt
```

Parametrelerin İşlevleri:

- **-d:** Taraması yapılacak olan ana alan adını (root domain) belirler.
- **-o:** Tespit edilen tüm alt alan adlarını belirtilen bir metin dosyasına kaydeder.

Örnek Çıktı

Tarama tamamlandığında, hedef sisteme dair şu tip kritik noktalar listelenebilir:

- admin.example.com
- mail.example.com
- Vpn.example.com

Vhost Tarama (Virtual Host Enumeration)

Aynı IP adresi üzerinde birden fazla web sitesinin barındırıldığı (Virtual Hosting) senaryolarda, bazı siteler halka açık DNS kayıtlarında görünmez. Vhost tarama, bu "gölgelerdeki" siteleri bulma sanatıdır.

Mantık ve İşleyiş

Saldırganlar, sunucunun kapısını (IP adresi) çalar ve "Host" başlığına (Header) farklı isimler yazarak içeride başka kimlerin yaşadığını sorarlar. Eğer sunucu bu isme özel bir yanıt dönerse, gizli bir Vhost tespit edilmiş olur.

Profesyonel Araç: ffuf

ffuf, bu işlemi otomatize etmek için en hızlı ve esnek araçtır.

Örnek Komut:

```
ffuf -u https://example.com -H "Host: FUZZ.example.com" -w /path/to/wordlist.txt
```

- -u: Ana hedef URL'yi (sunucu kapısını) belirtir.
- -H: Host başlığına her defasında listeden yeni bir isim (FUZZ) yerleştirir.
- -w: Denenecek potansiyel vhost isimlerinin bulunduğu sözlüğü belirtir.

Örnek Çıktı: Tarama sonunda dev.example.com (geliştirme), staging.example.com (hazırlık) veya admin.example.com (yönetim) gibi kritik ve gizli alanlar gün yüzüne çıkabilir.



Gobuster: Vhost Tarama Modu

Gobuster, sadece dizin taramakla kalmaz; aynı zamanda hedef sunucudaki **Vhost (Virtual Host)** yapılandırmalarını keşfetmek için de son derece hızlı ve kararlı bir araçtır.



Çalışma Mekanizması

Gobuster'ın vhost modu, hedef URL'ye gönderilen isteklerin başlık (header) kısmını kelime listesindeki verilerle değiştirerek sunucunun gizli kimliklerini açığa çıkarır.



Temel Kullanım Örneği

Sunucuda tanımlı sanal ana bilgisayarları tespit etmek için şu komut yapısı kullanılır:

Bash

```
gobuster vhost -u https://example.com -w /path/to/wordlist.txt
```

Komut Parametreleri:

- **vhost**: Sanal ana bilgisayar tarama modunu aktif hale getirir.
- **-u**: Taramaya baz teşkil edecek ana URL'yi belirtir.
- **-w**: Denenecek isimlerin bulunduğu sözlük (wordlist) dosyasının yolunu gösterir.



Örnek Çıktı Senaryosu

Bu tarama sayesinde, standart bir DNS sorgusunda çıkmayabilecek şu kritik noktalar tespit edilebilir:

- `api.example.com`
- `admin.example.com`
- `test.example.com`

Siber_Security Strateji Notu

ffuf mu, Gobuster mı? Her iki araç da Vhost taramasında mükemmeldir. Ancak Gobuster, özellikle çıktıları daha düzenli raporladığı ve vhost modu doğrudan optimize edildiği için "hızlı keşif" aşamasında büyük avantaj sağlar.

GET Parametre Fuzzing: Uygulama Mantığını Sorgulamak

Web uygulamaları, URL üzerindeki parametreler aracılığıyla sunucuya veri gönderir (Örn: id=5, user=admin). GET parametre fuzzing, bu değerlerin yerine beklenmedik veya zararlı girdiler göndererek sistemin tepkisini ölçme sanatıdır.

Amacı ve İşleyişi

Bu test yöntemiyle uygulamanın;

- Girdileri düzgün doğrulayıp doğrulamadığı,
- Beklenmedik girdiler karşısında hata mesajları (Error-based leakage) verip vermediği,
- Hassas verilere erişim imkanı tanıyıp tanımadığı analiz edilir.

Öne Çıkan Araçlar

1. ffuf (Hızlı ve Esnek)

ffuf, URL parametrelerini saniyeler içinde binlerce farklı payload ile test edebilir.

Örnek Kullanım:

```
ffuf -u "https://example.com/page.php?param=FUZZ" -w /path/to/wordlist.txt
```

- **-u:** Test edilecek URL'yi belirler. FUZZ anahtar kelimesi, payload'ların yerleştirileceği noktadır.
- **-w:** Test edilecek değerlerin bulunduğu kelime listesini belirtir.

2. Burp Suite Intruder (Güçlü ve Görsel)

Endüstri standardı olan Burp Suite, yakalanan istekler üzerinde çok daha detaylı analiz imkanı sunar.

Kullanım Adımları:

1. **İsteği Yakala:** Proxy üzerinden geçen GET isteğini Intruder sekmesine gönderin.
2. **İşaretle:** Test edilecek parametreyi (örn: ?id=1) hedef olarak belirleyin.
3. **Saldır:** Kelime listesini (payloads) yükleyin ve testi başlatın.

Siber_Security Notu

Neye Bakmalı? Fuzzing yaparken sadece "200 OK" koduna odaklanmayın. Yanıt boyutundaki (Length) küçük değişimler veya yanıt süresindeki (Time) gecikmeler, içeride bir yerlerde bir şeylerin değiştiğinin (örneğin başarılı bir sızıntının) habercisi olabilir.

POST Parametre Fuzzing: Perde Arkasındaki Veriyi Sorgulamak

POST parametre fuzzing, web uygulamalarına gönderilen (ve URL'de gizli kalan) verilerin manipüle edilerek güvenlik açıklarının keşfedilmesidir. Özellikle kimlik doğrulama formları ve veri girişi yapılan alanlar bu saldırının ana hedefidir.

Amacı ve İşleyişi

Saldırganlar, POST istekleriyle iletilen kritik verilerin uygulamanın işleyişini bozup bozmadığını veya hassas verilere

(kullanıcı hesapları gibi) erişim sağlayıp sağlamadığını analiz eder. Uygulamanın verdiği hata mesajları veya beklenmedik yanıtlar, bir bilgi sızıntısının habercisidir.

Örnek Bir HTTP İsteği Analizi:

HTTP

POST /login.php HTTP/1.1

Host: example.com

Content-Type: application/x-www-form-urlencoded

username=admin&password=123456

Buradaki username ve password alanları, otomatize araçlarla binlerce farklı kombinasyonla test edilerek sistemin zayıflığı ölçülür.

Profesyonel Araçlar ve Uygulama

1. Burp Suite Intruder (Kapsamlı Analiz)

POST isteklerini analiz etmek için endüstri standardıdır.

- **Kullanım:** Proxy ile yakalanan istek Intruder'a gönderilir, parametreler (password=FUZZ gibi) işaretlenir ve belirlenen kelime listesiyle saldırı başlatılır. Yanıtlar; durum kodları, yanıt süresi ve içerik bazlı olarak titizlikle incelenir.

2. ffuf (Yüksek Hızlı Fuzzing)

Hızın kritik olduğu durumlarda POST istekleri için de mükemmel bir tercihtir.

Örnek Komut:

Bash

```
ffuf -u "https://example.com/login.php" -X POST -d
```

```
"username=admin&password=FUZZ" -w /usr/share/wordlists/rockyou.txt -
```

```
H "Content-Type: application/x-www-form-urlencoded"
```

- **-X POST:** İstek yöntemini POST olarak belirler.

- **-d:** Gönderilecek form verilerini ve FUZZ noktasını tanımlar.
- **-H:** Gönderilen verinin tipini (Content-Type) sunucuya bildirir.

💡 Siber_Security Strateji Notu

Güvenlik Uyarısı: POST parametreleri URL'de görünmediği için genellikle "daha güvenli" sanılır. Ancak siber güvenlikte "gizlilik" (obscurity), "güvenlik" (security) değildir. Düzgün doğrulanmayan her POST parametresi, sisteminize sızmak için bekleyen açık bir kapıdır.

🔗 Captcha Bypass Brute Force (Güvenlik Mekanizmasını Atlatma)

Captcha, temel amacı botları durdurmak ve sisteme sadece insanların eriştiğinden emin olmak olan bir güvenlik katmanıdır. Ancak zayıf tasarım veya hatalı uygulama, bu kalkanı saldırganlar için bir hedef haline getirir.

🔍 Captcha Neden ve Nasıl Atlatılır?

Captcha korumasını aşmaya yönelik saldırılar genellikle sistemin mantıksal hatalarına veya teknolojik yetersizliklerine odaklanır:

1. **Öngörülebilir Kodlar:** Captcha sistemi rastgele değil de belirli bir algoritma veya desen üzerinden (örneğin sadece 4 haneli sayılar) kod üretiyorsa, saldırgan bu kısıtlı havuzu brute-force ile kolayca tarayabilir.
2. **Yinelenen Kodlar:** Eğer sistem üretilen bir kodu geçersiz kılmak yerine belirli aralıklarla tekrar kullanıyorsa, saldırgan tespit ettiği bu kodları kullanarak sisteme sızabilir.
3. **Makine Öğrenimi ve OCR:** Görsel tabanlı captcha'lar, optik karakter tanıma (OCR) veya özel eğitilmiş makine öğrenimi modelleri kullanılarak otomatik olarak okunabilir ve botlar tarafından geçilebilir.

🔗 Captcha Bypass: Uygulama Üzerinden Analiz

Captcha, otomasyonu durdurmak için tasarlanmıştır. Ancak aşağıdaki gibi bir HTTP isteği yapısında, captcha doğrulaması sunucu tarafında zayıf kurgulanmışsa saldırıya davetiye çıkarır.

🔍 Örnek Senaryo Analizi

Bir web uygulamasında giriş yaparken sunucuya gönderilen şu isteği inceleyelim:

HTTP

POST /login.php HTTP/1.1

Host: example.com

Content-Type: application/x-www-form-urlencoded

username=testuser&password=password123&captcha=1234

Zafiyet Noktaları:

- **Tahmin Edilebilirlik:** Eğer captcha=1234 değeri her zaman 4 haneli rakamlardan oluşuyorsa, bir saldırgan ffuf veya Burp Suite kullanarak 0000-9999 arasındaki tüm sayıları saniyeler içinde deneyebilir.
- **Statik Değer:** Eğer captcha bir kez doğru girildikten sonra oturum (session) boyunca değişmiyorsa, saldırgan doğru captcha'yı bir kez bulup ardından sadece password kısmına brute-force yapabilir.
- **Client-Side Validation:** Eğer captcha doğrulaması sadece tarayıcıda yapıp sunucuya (backend) sorulmuyorsa, saldırgan bu parametreyi isteğinden tamamen silebilir veya manipüle edebilir.

🔗 Captcha Solver Araçları ve Bypass Teknikleri

Captcha her ne kadar botları engellemek için tasarlanmış olsa da, otomatize araçlar bu engeli aşmak için sürekli gelişmektedir. Hedef sistemin zayıflığına göre kullanılan başlıca yöntemler şunlardır:

1. Captcha Solver (Makine Öğrenimi)

Gelişmiş Captcha sistemlerini (görsel tanıma, karmaşık karakterler vb.) aşmak için makine öğrenimi tabanlı tanıma araçları kullanılır. Bu araçlar, görseldeki karakterleri yüksek doğrulukla metne dönüştürerek botların sisteme giriş yapmasını sağlar.

2. Burp Suite Intruder ile Brute Force

Web formlarını brute-force ile taramak için kullanılan en esnek güvenlik testi aracıdır. Captcha mekanizması zayıfsa, şu adımlarla devre dışı bırakılabilir:

- **İsteği Yakalama:** Proxy üzerinden ilgili giriş isteği yakalanır ve Intruder sekmesine gönderilir.
- **Hedefleme:** Captcha alanı hedef olarak seçilir ve payload listesine uygun tahmin değerleri eklenir.
- **Analiz:** Saldırı başlatıldıktan sonra dönen HTTP yanıtları incelenerek başarılı girişler tespit edilir.

Korunma Yöntemleri: Kaleyı Nasıl Savunuruz?

Saldırganların bu yöntemlerini etkisiz hale getirmek için çok katmanlı bir savunma stratejisi izlenmelidir:

- **Güçlü Captcha Sistemleri:** Öngörülebilir olmayan, rastgele üretilen ve makine öğrenimiyle tanınması zor olan (reCAPTCHA v3 gibi) sistemler tercih edilmelidir.
- **Ek Güvenlik Katmanları:** Sadece Captcha'ya güvenilmemeli; iki aşamalı doğrulama (2FA) veya şüpheli IP adreslerini engelleme gibi ek önlemler uygulanmalıdır.
- **Kullanıcı Davranış Analizi:** Anormal oturum açma denemelerini veya bot davranışlarını tespit etmek için imza tabanlı değil, davranış tabanlı (UBA) analizler kullanılmalıdır.

Siber_Security Final Notu

"Captcha bir zırh değil, sadece bir hız tümseğidir. Eğer sisteminizde davranış analizi ve IP tabanlı kısıtlamalar yoksa, en karmaşık Captcha bile yeterli işlemci gücü ve doğru araçlarla aşılabılır. Gerçek güvenlik, katmanlı savunmadan geçer!"

Soru 1

Yaygın olarak kullanılan parola listelerinin sistematik bir şekilde denenmesi ile yapılan saldırı türü hangisidir?

Credential Attack

Dictionary Attack

Brute Force Attack

Hybrid Attack

Soru 2

Daha önce veri ihlali yaşanmış ve internete sızmış kimlik bilgilerini kullanarak farklı web sitelerinde oturum açılmanın denendiği saldırı türü hangisidir?

Brute Force Attack

Dictionary Attack

Hybrid Attack

Credential Attack

Soru 3

Aşağıdakilerden hangisi brute force saldırılarının etkilerinden değildir?

Hesap Ele Geçirme

Yetkisiz Erişim

Uzaktan Komut Çalıştırma

Hizmet Kesintisi

Soru 4

gobuster aracında dizin taraması yapılmak istendiğini belirtmek için hangi anahtar kelime kullanılır?

DIR

Soru 5

ffuf aracında hedef url belirtmek için hangi parametre kullanılır? (Cevap formatı: -x)

-u

Soru 6

ffuf aracında kelime listesi belirtmek için hangi parametre kullanılır? (Cevap formatı: -x)

-w

Soru 7

ffuf aracını kullanırken hangi alana fuzzing uygulanacağını belirtmek için kullanılan anahtar kelime nedir?

Fuzz

Soru 8

ffuf aracında POST istek yöntemi ile bir veri gönderilmek istendiğinde form verisi hangi parametre ile belirtilir? (Cevap formatı: -x)

-d

Soru 9

ffuf aracında HTTP istek yöntemi belirtmek hangi parametre kullanılır? (Cevap formatı: -X)

